

Can We Derive Agile Metrics from Raw GitHub Data?

Aishwarya Pujitha · Apoorva · Vansh Vekaria · Soorya Vasudevan

Motivation & Research Problem

Agile teams measure velocity and productivity to plan sprints - but no tool automatically derives these metrics from version control data.

The **AIDev** dataset contains thousands of real GitHub pull requests by human developers. We ask: can we mine raw commit data to reconstruct meaningful agile metrics - without any manual story-point estimation?

Research Questions

- RQ1** Can agile story sizes be derived from PR timestamps, and how does effort vary by task type?
- RQ2** Can distinct developer productivity profiles be identified through unsupervised clustering?
- RQ3** Can a PR's size be predicted from its title before work begins?
- RQ4** How accurately can PR cycle time be predicted, and what signals drive it?

Dataset & Pipeline

AIDev (hao-li/AIDev · HuggingFace)
human_pull_request + human_pr_task_type

- Join PRs on id · compute dev_time_hrs = closed_at - created_at
- Drop open PRs (no closed_at) · remove instant closes ≤ 5 min
- Remove bot users · deduplicate on (user, title)
- Per-type IQR outlier removal across 11 task types
- Quartile binning per type $\rightarrow S/M/L/XL \rightarrow$ effort points 1/2/4/8

Story Point Derivation - Key Design

Traditional agile teams estimate story points through planning poker - a human, consensus-driven process. AIDev has no such estimates, so we construct them. For each of the **11 task types independently**, PRs are ranked by development time and split into four quartiles: $S/M/L/XL$, mapped to a doubling scale (1/2/4/8). Per-type binning is deliberate: feature PRs routinely take 3-5 $\times$ longer than CI changes, so a single global threshold would systematically mislabel all CI work as 'small.' This approach is a transparent, reproducible proxy - and one that **requires no human estimation whatsoever**.

Use Case 1: Effort Trend Analysis

Monthly avg dev time + avg effort points per contributor (≥ 2 active months). Dual-axis interactive chart; contributors tagged with productivity bucket badge.

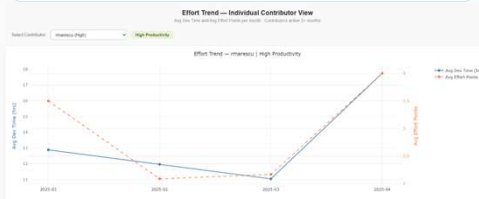


Fig 1. Effort trend - 'lmarescu' (High Productivity). Blue = avg dev time (hrs); orange dashed = avg effort points.

Use Case 1 Results - Effort Distribution by Type

After per-type IQR cleaning, each type shows a tighter distribution. Feature PRs take longest; CI/style PRs are far shorter - justifying per-type binning over global quartiles.

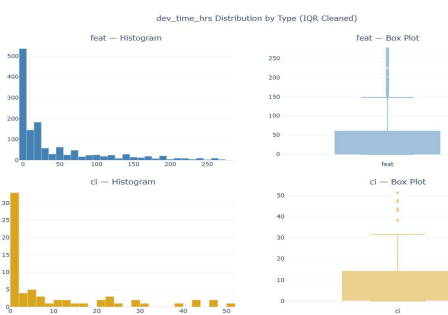


Fig 2. dev_time_hrs by type after IQR cleaning (feat, fix, chore, docs, refactor, build shown). Outliers removed; scales now comparable.

Use Case 2: Productivity Clustering - K-Means

Features: avg_dev_time_hrs · avg_effort_points · avg_monthly_velocity · merge_rate · productivity
Validation: Elbow + silhouette (k=2-40) · DBSCAN cross-validation

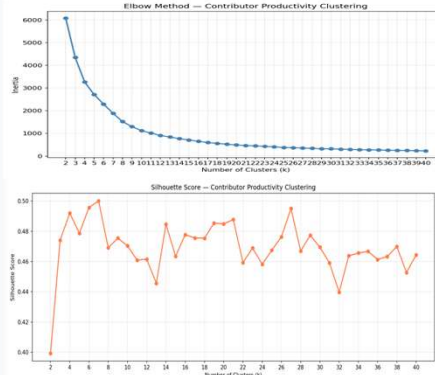


Fig 3. Elbow method (upper) confirms diminishing returns beyond k=7. Silhouette score (lower) peaks at k=7 (score = 0.50), indicating optimal and well-separated cluster structure.

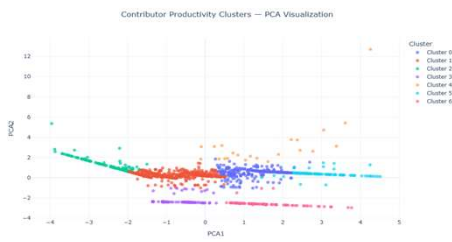


Fig 4. PCA visualization of K-Means clusters (k = 7). Clear separation confirms distinct productivity profiles.

From 7 Clusters $\rightarrow$ 5 Productivity Buckets
Each cluster's mean productivity score is computed, then mapped to a named bucket - bridging abstract cluster IDs to interpretable labels.

Kmeans Cluster	Prod. Mean	Bucket
6	0.0015	Very Low
3	0.0121	Low
5	0.0630	Low
0	0.2384	Medium
1	0.9089	High
4	2.0900	High
2	6.5563	Very High

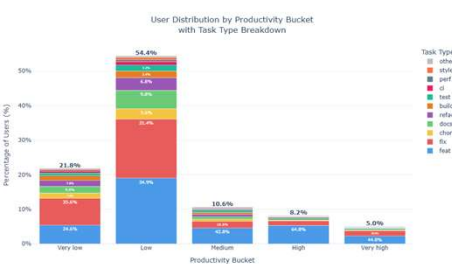


Fig 5. User distribution by productivity bucket + task type. High-productivity contributors skew toward feature (feat) PRs.

Use case 2 Finding: Distinct productivity profiles exist and are stable under both K-Means and DBSCAN. 54.4% of contributors fall in the 'low' bucket; high-productivity contributors take 44.8 - 64.8% feature PRs or chore/fix-heavy work in lower buckets.

Use Case 3: Auto Effort Estimation from PR Title

Goal: predict a PR's S/M/L/XL story size from creation-time signals only - title & body text, task type, temporal features.
Protocol: no-leakage rule excludes closed_at / dev_time · 3,998 PRs · 80/20 stratified split · ordinal S<M<L<XL scored by QWK & macro-F1.

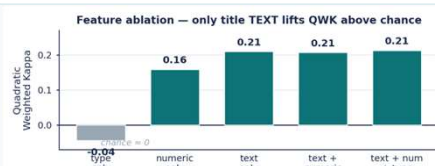


Fig 6. Feature ablation (QWK, 5-fold CV). Task type alone = chance; only title text lifts QWK above 0 - confirming per-type binning removes type as a shortcut.

UC3 Finding: a PR title carries a weak but statistically real size signal. Best model (XGBoost) on 800 held-out PRs: balanced acc 0.33 (chance 0.25), QWK 0.20 [95% CI 0.30-0.36]; 70% of predictions land within one size class. Temporal (future-PR) split: QWK 0.17. Negative control: adding dev_time + type drives accuracy to 98.5%, proving the label is timestamp-deterministic and rightly excluded. Title text is a useful size prior, not a replacement for measured cycle time.

Use Case 3 Approach & Model Bake-off

- **Leakage audit** - keep only signals known when a PR opens (exclude closed_at / dev_time).
- **Features** - title/body TF-IDF text · text stats · task type · creation time.
- **Preprocessing** - sparse TF-IDF (linear/NB); SVD-dense (trees, boosting, MLP, KNN).
- **Evaluation** - 5-fold CV bake-off + random/temporal splits + leakage control.

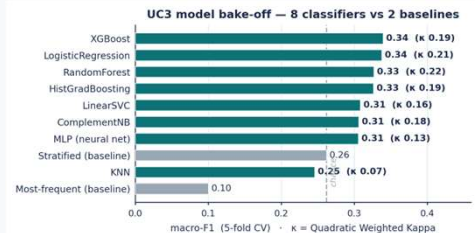


Fig 7. UC3 model bake-off - 5-fold CV macro-F1 (κ = QWK). Learners cluster tightly just above chance.

Use Case 4: Cycle Time Regression & Forecasting

Models: Prophet (time-series) · XGBoost (gradient boosting)
Target: dev_time_hrs (cycle time) · Eval: RMSE on held-out test set · 95% CI shown per model

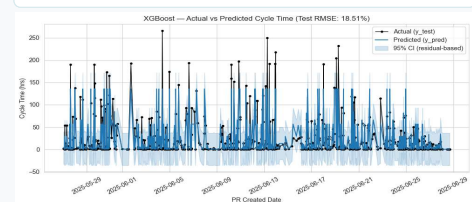


Fig 8. XGBoost - Actual vs Predicted Cycle Time (Test RMSE: 18.51%).

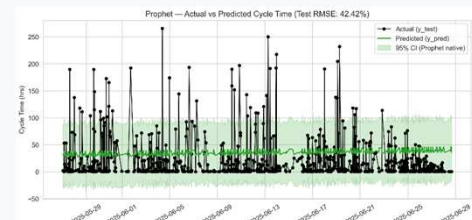


Fig 9. Prophet - Actual vs Predicted Cycle Tim (Test RMSE: 42.42%).

UC4 Finding: XGBoost (RMSE 18.51%) closely tracks actual cycle times by leveraging PR-level features, far outperforming Prophet's flat, seasonality-only predictions (RMSE 42.42%). This confirms cycle time is driven mainly by individual PR characteristics rather than temporal trends, making XGBoost the stronger model for this use case.

What We Learned

- Per-type IQR cleaning is essential - global IQR masks that feature PRs naturally take 3-5 $\times$ longer than CI changes.
- Open PRs = cycle time. They are in-progress WIP. Excluding them is standard agile practice, not a data quality decision.
- Productivity without context misleads - the bucket + task type breakdown reveals who does hard work vs trivial fixes.
- Limitation: Observation window ends at PR close - local revisions before first push are invisible.

How AI Helped (and Didn't)

- Helped** Architecture, pipeline design, and all analytical decisions were ours. AI assisted with pandas syntax and fixing minor code errors
- Helped** For the UC1 interactive HTML chart, AI helped with JavaScript/Plotly.js syntax. The dual-axis design, dropdown, and productivity badge were our idea
- Didn't** AI could not replace human judgment on threshold choices (5-min cutoff, k = 7 selection, 5-bucket productivity split).
- Didn't** On UC3, no model AI suggested could push title-only size prediction far past chance (QWK = 0.2) - AI cannot manufacture predictive signal the data does not contain.

Implications & Ethics

Practice: Engineering teams can auto-generate sprint velocity baselines from GitHub history without manual estimation.

Ethics: Productivity metrics from commit data must NOT be used for performance reviews. They reflect task assignment, not individual capability. Team-level aggregation only.

Research: This pipeline is fully reproducible on any PR dataset. The method generalises beyond AIDev to any team using GitHub for version control.